JAIN DEEMED-TO-BE UNIVERSITY | ONLINE

Self-Learning Material

Program:MCA

Semester: 4

Course Name: Web APIs

Code: 21VMT3S402

Unit Name: Spring Boot View

Table of Contents

Objectives:

In this Unit you will learn to –

- Gain a comprehensive understanding of Thymeleaf.
- Explore the various features and capabilities offered by Thymeleaf, such as dynamic content rendering, template inheritance, data binding, and integration with Spring Boot.
- Dive into Thymeleaf's syntax, including its expression language.
- Learn to create dynamic and data-driven web pages using Thymeleaf

Learning Outcome:

Learn to use the ThymeLeaf Template Engine.

13. Spring Boot View

1. Introduction to Thymeleaf

Thymeleaf is a Java-based template engine that simplifies the process of creating dynamic and interactive web pages. It is particularly well-suited for use with Spring Boot applications but can also be integrated into standalone Java applications. Thymeleaf's main objective is to bridge the gap between the backend and frontend by enabling developers to seamlessly integrate server-side logic and dynamic data into their templates.

1. Natural Templates: Thymeleaf templates closely resemble standard HTML files. This makes them intuitive and easy to work with, even for developers who are not familiar with Thymeleaf. You can view and edit Thymeleaf templates directly in a web browser or an HTML editor.

2. Expression Language (Thymeleaf Standard Expression Language - ${...}): Thymeleaf offers a powerful expression language that allows you to manipulate and process data directly within your templates. You can use expressions to access variables, perform arithmetic operations, manipulate strings, and more.

3. Data Binding: Thymeleaf simplifies the process of binding data from the backend to the frontend. You can easily populate your templates with dynamic data by evaluating expressions and displaying values from Java objects.

4. Layouts and Fragments: Thymeleaf encourages the use of reusable layout templates and fragments. This promotes code reusability and maintainability by allowing you to define common parts of your web pages (like headers, footers, and navigation menus) in separate templates and include them where needed.

5. Conditional Rendering: With Thymeleaf, you can conditionally render elements and content based on specific conditions. This is particularly useful for displaying different content based on user roles, states, or any other dynamic factors.

6. Iterating Over Collections: Thymeleaf provides mechanisms for iterating over

collections such as lists, maps, and arrays. You can dynamically generate HTML elements based on the elements in a collection, making it easy to display dynamic lists or tables.

7. Form Handling: Thymeleaf simplifies the process of creating and processing forms in web applications. It generates form fields, handles form submissions, and binds form data to Java objects, reducing the amount of boilerplate code required.
8. Internationalization and Localization: Thymeleaf supports internationalization (i18n) and localization (l10n) through its message resolution mechanism. This enables the creation of multi-language web applications.

Advantages:

- Seamless Integration: Thymeleaf integrates seamlessly with Spring Boot, making it an excellent choice for Spring-based applications.
- Reduced Development Time: The natural syntax and built-in features of Thymeleaf streamline the development process and reduce the amount of code you need to write.
- Enhanced Collaboration: Thymeleaf's approach of mixing regular HTML with expressions allows frontend and backend developers to work collaboratively without steep learning curves.
- Consistent User Experience: Thymeleaf helps ensure a consistent user experience by enabling dynamic content generation and rendering on the server side.

Thymeleaf simplifies the creation of dynamic and data-driven web pages by providing a natural and powerful template engine that seamlessly integrates with Java applications, especially Spring Boot projects. Its features and benefits make it a valuable tool for developers aiming to build modern, interactive, and maintainable web applications.

2. Need of Thymeleaf

The need for Thymeleaf arises from the challenges and requirements faced by developers when building dynamic web applications. Thymeleaf addresses these needs by providing a powerful and intuitive template engine that simplifies the integration of dynamic data and server-side logic into web templates. Here are some key reasons why Thymeleaf is needed:

1. Dynamic Content Generation: Traditional web applications often require dynamically generated content based on user interactions, database queries, or other backend processes. Thymeleaf enables the seamless integration of dynamic data into HTML templates, allowing developers to generate content on the server and deliver fully-rendered pages to the client.

2. Integration with Backend Logic: Developers often need to incorporate server-side logic and data processing into web pages. Thymeleaf's expression language allows embedding of Java code directly into templates, making it easier to execute backend operations and manipulate data before rendering.

3. User Interface Consistency: Thymeleaf promotes the use of reusable layout templates and fragments. This ensures a consistent user interface across different pages of the application, as common elements like headers, footers, and navigation menus can be defined once and reused throughout the application.

4. Data Binding and Presentation: Web applications frequently need to display data from backend sources such as databases or APIs. Thymeleaf simplifies data binding by allowing developers to bind Java objects to template expressions, making it straightforward to present dynamic data to users.

5. Form Handling: Forms are a crucial part of web applications for user input and data submission. Thymeleaf provides mechanisms to generate form fields, bind form data to Java objects, and handle form submissions, reducing the complexity of form-related code.

6. Conditional Rendering: Web pages often need to display content conditionally based on various factors, such as user roles or states. Thymeleaf's conditional constructs enable developers to control the rendering of content based on these conditions.

7. Simplifying Frontend Development: Thymeleaf shifts some of the rendering and templating responsibilities from the frontend to the backend, allowing frontend developers to focus more on designing and styling the user interface rather than worrying about data integration and manipulation.
8. Internationalization and Localization: Applications targeting a global audience require support for multiple languages and cultures. Thymeleaf's message resolution mechanism facilitates internationalization and localization efforts by dynamically rendering content based on the user's preferred language.
9. Easy Learning Curve: Thymeleaf's syntax closely resembles standard HTML, making it approachable for both frontend and backend developers. This reduces the learning curve and allows developers to quickly adopt and utilize Thymeleaf in their projects.
10. Seamless Spring Boot Integration: Thymeleaf is the default templating engine for Spring Boot applications. It is designed to work seamlessly with Spring's ecosystem, offering straightforward integration and leveraging the benefits of Spring's dependency injection and other features.

In essence, Thymeleaf fulfills the need for a flexible, feature-rich, and developer-friendly template engine that empowers developers to create dynamic, data-driven, and interactive web applications with ease. Its ability to bridge the gap between the backend and frontend makes it a valuable tool for building modern web solutions.

3. Features of Thymeleaf

Thymeleaf is a robust and versatile template engine that offers a wide range of features to simplify the process of creating dynamic web pages in Java-based applications. Its features empower developers to seamlessly integrate dynamic data and server-side logic into their templates.

Here are some of the key features of Thymeleaf:

1. Natural Templates: Thymeleaf templates use standard HTML syntax, making

them easy to read, write, and maintain. Developers can work with these templates using common HTML editors and browsers.

2. Expression Language (Thymeleaf Standard Expression Language - ${...}): Thymeleaf provides a powerful expression language that allows developers to access and manipulate data within templates. Expressions can be used to render variables, perform arithmetic operations, access attributes, and more.

3. Data Binding: Thymeleaf simplifies the process of binding data from the backend to the frontend. It supports data binding by evaluating expressions and rendering dynamic content based on Java objects or other backend sources.

4. Layouts and Fragments: Thymeleaf encourages the creation of reusable layout templates and fragments. Layouts help define the overall structure of a web page, while fragments allow developers to encapsulate and reuse specific parts of the page, such as headers, footers, and navigation menus.

5. Conditional Rendering: Thymeleaf supports conditional rendering, enabling developers to control the visibility of elements or content based on specific conditions. This feature is useful for creating dynamic user interfaces that adapt to different scenarios.

6. Iteration and Loops: Thymeleaf simplifies the iteration over collections (lists, sets, arrays, etc.) by providing iteration constructs. Developers can easily generate repeated content, such as tables or lists, based on the elements in a collection.

7. Form Handling: Thymeleaf streamlines form creation and handling. It can generate form fields, populate form data from Java objects, and handle form submissions, reducing the amount of manual form-related code.

8. Internationalization and Localization: Thymeleaf supports internationalization (i18n) and localization (l10n) through its message resolution mechanism. Developers can define and display localized messages and content based on the user's preferred language.

9. Attribute Processing (th:attr): Thymeleaf allows developers to dynamically set HTML attributes on elements using expressions. This feature is helpful for managing attributes such as CSS classes, styles, and dynamic URLs.

10. Template Inclusion and Fragment Reuse: Thymeleaf enables developers to include templates within other templates, facilitating reuse and modularity. This is especially useful for components shared across different pages.

11. Standard Dialect: Thymeleaf provides a set of predefined attributes and tags, collectively known as the Standard Dialect. These include th:text, th:if, th:each, th:object, th:attr, and more, which simplify various aspects of template manipulation.

12. JavaScript Integration: Thymeleaf allows you to include JavaScript code within your templates, enabling dynamic client-side behavior while still leveraging server-side data.

13. Processing Text Templates: Apart from HTML and XML templates, Thymeleaf can also process plain text templates, making it versatile for generating various types of content.

14. Extensibility: Thymeleaf is designed to be extensible, allowing developers to create custom dialects and processors to cater to specific requirements.

These features collectively make Thymeleaf a powerful tool for creating dynamic and interactive web applications that seamlessly integrate backend logic and data with frontend templates. Thymeleaf's user-friendly syntax and extensive feature set contribute to its popularity in the Spring Boot ecosystem and beyond.

4. Template Types - Thymeleaf Support

Thymeleaf supports various template types, allowing developers to generate dynamic content for different purposes and formats. Its flexibility makes it suitable for a wide range of use cases.

Here are the template types that Thymeleaf supports:

- HTML Templates: Thymeleaf is commonly used to create dynamic HTML templates for web applications. With Thymeleaf's natural template syntax, developers can seamlessly integrate dynamic data and server-side logic into HTML documents. This includes rendering variables, applying conditional logic, iterating over collections, and more.

- XML Templates: Thymeleaf can also generate dynamic XML templates. This is useful for generating XML-based documents, such as RSS feeds, sitemaps, configuration files, or any other XML content required by the application.
- Text Templates: Thymeleaf is not limited to generating markup-based templates. It can also generate plain text templates, making it suitable for generating emails, notifications, reports, and other types of text-based content.
- JavaScript Templates: While not a separate template type, Thymeleaf can be used to generate JavaScript code within HTML templates. This is helpful for embedding dynamic client-side behavior directly in the template, using server-side data.
- CSV/Excel Templates: Thymeleaf can be creatively used to generate comma-separated values (CSV) files or even Excel files by applying appropriate templating strategies. This is useful for generating reports or data exports.
- Other Formats: With its extensible architecture, Thymeleaf can potentially support other template types or formats by creating custom dialects and processors. This allows developers to tailor Thymeleaf to their specific needs.

Thymeleaf's ability to generate various template types offers developers the flexibility to generate content in formats beyond traditional HTML, making it a versatile tool for building dynamic and data-driven applications that require different types of output. The consistent syntax and feature-rich expression language make Thymeleaf a powerful choice for handling diverse template needs within Java-based applications.

5. Standard Dialect

The Thymeleaf Standard Dialect is a set of predefined attributes and tags that provide powerful functionality for manipulating templates, rendering dynamic content, and controlling the presentation of data. These features simplify the process of integrating dynamic data and server-side logic into your templates. The Standard Dialect is the default and most commonly used dialect in Thymeleaf, and it covers a

wide range of use cases. Here are some of the key features provided by the Thymeleaf Standard Dialect:

- th:text: The th:text attribute is used to replace the text content of an element with the evaluated expression. It allows you to dynamically set the text value of an element based on a variable or an expression.
- th:if/th:unless: These attributes enable conditional rendering of elements. You can use th:if to conditionally include an element based on a condition, and th:unless to conditionally exclude an element.
- th:each: The th:each attribute is used to iterate over collections (lists, sets, arrays, etc.) and render content for each element. It simplifies the process of generating repetitive content, such as tables or lists.
- th:object: The th:object attribute specifies the object that should be used as a data context for expressions within a specific element or block. It allows you to set a temporary variable that will be used as the base for expression evaluation.
- th:attr: The th:attr attribute dynamically sets HTML attributes on elements. This is useful for applying CSS classes, inline styles, or any other attribute based on expressions or conditions.
- th:fragment: The th:fragment attribute defines reusable template fragments. These fragments can be included in other templates using the th:insert or th:replace attributes.
- th:replace/th:insert: These attributes allow you to include template fragments defined using th:fragment in other templates. th:replace replaces the entire element with the fragment, while th:insert includes the fragment within the element.
- th:href/th:src: The th:href and th:src attributes dynamically set the href and src attributes of anchor (<a>) and image (<img>) elements, respectively, based on expressions.
- th:class/th:style: The th:class and th:style attributes dynamically set CSS classes and inline styles on elements, respectively, based on expressions.

- th:switch/th:case: These attributes are used for switch-case-like scenarios. th:switch defines the switch expression, while th:case defines individual cases that are evaluated based on the switch expression.
- th:text="${...}": The expression syntax ${...} allows you to interpolate values from Java objects into the template, dynamically replacing placeholders with actual values.

These are just a few examples of the features provided by the Thymeleaf Standard Dialect. Each of these attributes and tags enhances the templating capabilities of Thymeleaf and empowers developers to create dynamic and interactive web pages with ease. The Standard Dialect is the foundation of Thymeleaf's templating functionality and is widely used in Spring Boot applications and other Java-based projects.

6. Creating a Basic Thymeleaf Template

In this hands-on exercise, we will walk you through the process of creating a basic Thymeleaf template step by step. You'll learn how to set up a simple Spring Boot application and create a Thymeleaf template to display dynamic content.

Step 1: Set Up a Spring Boot Project

- Open your preferred Java IDE (Eclipse, IntelliJ IDEA, etc.).
- Create a new Spring Boot project with Maven or Gradle.
- Add the necessary dependencies: Spring Web and Thymeleaf.

Step 2: Create a Controller

- Create a new Java class named HomeController (or any suitable name).
- Annotate the class with @Controller to indicate that it's a Spring MVC controller.
- Define a method to handle the home page request:

```
import org.springframework.stereotype.Controller;
```

```
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;


@Controller
public class HomeController {


    @GetMapping("/")
    public String home(Model model) {
        String message = "Welcome to Thymeleaf!";
        model.addAttribute("message", message);
        return "home";
    }
}
```

Step 3: Create a Thymeleaf Template

- Inside the resources/templates folder, create a new HTML file named home.html.
- Open the home.html file and add the following content:

```
<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <title>Thymeleaf Example</title>
</head>
<body>
    <h1 th:text="${message}"></h1>
</body>
</html>
```

Step 4: Run the Application

- Run the Spring Boot application.
- Open a web browser and navigate to http://localhost:8080.
- You should see the message "Welcome to Thymeleaf!" displayed on the page.

7. Building a Dynamic User Profile Page

In this exercise, we'll guide you through the process of building a dynamic user profile page using Thymeleaf and Spring Boot. You'll learn how to create a user profile template and populate it with dynamic data from a Java object.

Step 1: Set Up the Project

- Open your Java IDE and create a new Spring Boot project.
- Add the necessary dependencies: Spring Web and Thymeleaf.

Step 2: Create User Model

- Create a Java class named User to represent user data:

```
public class User {
    private String username;
    private String fullName;
    private String email;

    // Getters and setters
}
```

Step 3: Create a Controller

- Create a new Java class named ProfileController (or any suitable name).
- Annotate the class with @Controller.
- Define a method to handle the profile page request:

```
import org.springframework.stereotype.Controller;

import org.springframework.ui.Model;

import org.springframework.web.bind.annotation.GetMapping;


@Controller

public class ProfileController {


    @GetMapping("/profile")

    public String userProfile(Model model) {

        User user = new User();

        user.setUsername("johndoe");

        user.setFullName("John Doe");

        user.setEmail("johndoe@example.com");

        model.addAttribute("user", user);

        return "profile";

    }

}
```

Step 4: Create Thymeleaf Template

- Inside the resources/templates folder, create a new HTML file named profile.html.
- Open the profile.html file and add the following content:

```
<!DOCTYPE html>
```

```
<html lang="en" xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <title>User Profile</title>
</head>
<body>
    <h1>User Profile</h1>
                                    <p><strong>Username:</strong>         <span
 th:text="${user.username}"></span></p>
                                <p><strong>Full       Name:</strong>       <span
 th:text="${user.fullName}"></span></p>
    <p><strong>Email:</strong> <span th:text="${user.email}"></span></p>
</body>
</html>
```

Step 5: Run the Application

- Run the Spring Boot application.
- Open a web browser and navigate to http://localhost:8080/profile.
- You should see the user profile page with the dynamic data populated from the User object.